

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Отчёт №3

Структуры данных

(по дисциплине «Алгоритмы и структуры данных»)

Выполнил:

Лабин Макар Андреевич,
АиСД 3.2, Р3231, 466449.

Проверил:

Смирнов Виктор Игоревич

ИТМО

г. Санкт-Петербург, Россия
2026

Содержание

1	Яндекс.Контест	1
1.1	Машинки	1
1.1.1	Описание решения	1
1.1.2	Асимптотическая сложность	1
1.1.3	Листинг кода	1
1.2	Гоблины и очереди	3
1.2.1	Описание решения	3
1.2.2	Асимптотическая сложность	3
1.2.3	Листинг кода	4
1.3	Менеджер памяти-1	5
1.3.1	Описание решения	5
1.3.2	Асимптотическая сложность	5
1.3.3	Листинг кода	6
1.4	Минимум на отрезке	8
1.4.1	Описание решения	8
1.4.2	Асимптотическая сложность	8
1.4.3	Листинг кода	9
2	Timus	10
2.1	War Games 2	10
2.1.1	Описание решения	10
2.1.2	Асимптотическая сложность	10
2.1.3	Листинг кода	10
2.2	White Streaks	12
2.2.1	Описание решения	12
2.2.2	Асимптотическая сложность	12
2.2.3	Листинг кода	13

1 Яндекс.Контест

1.1 Машинки

1.1.1 Описание решения

Введём для всякой игрушки x_i типа t **приоритет** p_i как расстояние $j - i$ до следующей игрушки x_j с типом t .

Воспользуемся жадным алгоритмом: будем содержать на полу игрушки, которые обладают наименьшим приоритетом. Если Петя захотел поиграть с игрушкой типа t' , который не представлен на переполненном полу, то найдём игрушку с наивысшим приоритетом и на её место добавляем желанную игрушку.

Такой подход гарантирует, что с пола убираются лишь те игрушки, с которыми Петя захочет играть позже всего. Это, в свою очередь, минимизирует количество действий со стороны Петинной мамы.

1.1.2 Асимптотическая сложность

Временная сложность решения — $\mathcal{O}(P \cdot \log(NK))$. Заполнение массива приоритетов происходит за линейно-логарифмическое время $\mathcal{O}(P \cdot \log N)$, поскольку приоритет вычисляется за логарифмическое время для каждой игрушки (используется красно-чёрное дерево для хранения ссылок на типы игрушек в исходном массиве).

Симуляция жадного алгоритма происходит также за линейно-логарифмическое время $\mathcal{O}(P \cdot \log K)$. Игрушка наивысшего приоритета ищется за константу при помощи красно-чёрного дерева, остальные операции (*удаления, добавления*) работают за логарифмическое время для каждой игрушки.

Пространственная сложность решения — линейная $\mathcal{O}(P + N + K)$. Массив игрушек материализуется, на его основе создаётся массив приоритетов. Отдельно используется множество и массив используемых игрушек, которые также занимают линейное количество памяти.

1.1.3 Листинг кода

```
1 #include <algorithm>
2 #include <cstdlib>
3 #include <iostream>
4 #include <map>
5 #include <set>
6 #include <vector>
7
8 namespace {
9     std::size_t INFTY = 5 * 1e5;
10
11     std::vector<std::size_t> CalculatePriorities(const std::vector<
12         std::size_t>& array) {
13         std::vector<std::size_t> priorities(array.size(), INFTY);
14         std::map<std::size_t, std::size_t> toy_history = {};
15         for (std::size_t i = 0; i < array.size(); i++) {
```

```

15     const std::size_t toy_type = array[i];
16     if (toy_history.count(toy_type) != 0U) {
17         priorities[toy_history[toy_type]] = i;
18     }
19     toy_history[toy_type] = i;
20 }
21 return priorities;
22 }
23
24 struct Toy {
25     std::size_t idx;
26     std::size_t type;
27     bool used;
28
29     bool operator==(const Toy& other) const {
30         return type == other.type;
31     }
32 };
33
34 struct CompareByPriority {
35     const std::vector<std::size_t>& priorities;
36
37     explicit CompareByPriority(const std::vector<std::size_t>& arr
38         ) : priorities(arr) {
39
40     }
41
42     bool operator()(const Toy& l_toy, const Toy& r_toy) const {
43         if (priorities[l_toy.idx] != priorities[r_toy.idx]) {
44             return priorities[l_toy.idx] > priorities[r_toy.idx];
45         }
46         return l_toy.type < r_toy.type;
47     }
48 };
49 // namespace
50
51 int main() {
52     std::size_t n_count = 0;
53     std::size_t k_count = 0;
54     std::size_t p_count = 0;
55
56     std::cin >> n_count >> k_count >> p_count;
57
58     std::vector<std::size_t> toys(p_count);
59     for (std::size_t i = 0; i < p_count; i++) {
60         std::cin >> toys[i];
61     }
62
63     std::size_t action_counter = 0;
64     const std::vector<std::size_t> priorities =
65         CalculatePriorities(toys);

```

```

64 auto cmp = CompareByPriority(priorities);
65 std::set<Toy, decltype(cmp)> floor(cmp);
66 std::vector<Toy> active_toys(n_count + 1);
67
68 for (std::size_t i = 0; i < p_count; i++) {
69     const Toy current_toy = {i, toys[i], true};
70     if (active_toys[toys[i]].used) {
71         floor.erase(active_toys[toys[i]]);
72         floor.insert(current_toy);
73         active_toys[toys[i]] = current_toy;
74         continue;
75     }
76     if (floor.size() == k_count) {
77         const Toy top = *floor.begin();
78         floor.erase(top);
79         active_toys[top.type].used = false;
80     }
81     floor.insert(current_toy);
82     active_toys[toys[i]] = current_toy;
83     action_counter++;
84 }
85 std::cout << action_counter;
86 return 0;
87 }

```

1.2 Гоблины и очереди

1.2.1 Описание решения

Воспользуемся двусвязным списком для симуляции задачи. Разделим очередь гоблинов на две части и будем поддерживать каждую так, чтобы длина первой части отличалась от второй не больше чем на один.

Если приходит гoblin *обычный*, то добавляем его в конец второй части очереди. При необходимости перемещаем первого гоблина второй части в конец первой.

Если приходит гoblin *привилегированный*, то добавляем его либо в конец первой части очереди, либо в начало второй части (*чтобы соблюдалось оговорённое соотношение*).

Если подходит очередь следующего гоблина, то достаём его из начала первой части очереди. При необходимости перемещаем первого гоблина из второй части очереди в конец первой части.

1.2.2 Асимптотическая сложность

Временная сложность решения — линейная $\mathcal{O}(N)$. В алгоритме используются операции двусвязного списка, которые выполняются за константное время для каждого гоблина.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. В алгоритме очередь гоблинов материализована, поэтому используется линейное количество памяти.

1.2.3 Листинг кода

```
1 #include <cstdint>
2 #include <iostream>
3 #include <list>
4
5 namespace {
6 void AddGoblin(
7     std::list<std::size_t>& first_part,
8     std::list<std::size_t>& second_part,
9     const std::size_t idx,
10    const bool is_privileged
11 ) {
12     if (first_part.empty() && second_part.empty()) {
13         first_part.push_back(idx);
14         return;
15     }
16     if (second_part.empty()) {
17         second_part.push_back(idx);
18         return;
19     }
20     if (is_privileged) {
21         if (first_part.size() == second_part.size()) {
22             first_part.push_back(idx);
23             return;
24         }
25         second_part.push_front(idx);
26         return;
27     }
28     second_part.push_back(idx);
29     if (first_part.size() < second_part.size()) {
30         const std::size_t middle_idx = second_part.front();
31         second_part.pop_front();
32         first_part.push_back(middle_idx);
33         return;
34     }
35 }
36
37 void PopGoblin(std::list<std::size_t>& first_part, std::list<std
38 ::size_t>& second_part) {
39     const std::size_t to_pop = first_part.front();
40     first_part.pop_front();
41     std::cout << to_pop << "\n";
42
43     if (first_part.size() < second_part.size()) {
44         const std::size_t middle_idx = second_part.front();
45         second_part.pop_front();
46         first_part.push_back(middle_idx);
47     }
48 }
49 } // namespace
```

```

49
50 int main() {
51     std::size_t n_count = 0;
52     std::cin >> n_count;
53
54     std::list<std::size_t> first_part = {};
55     std::list<std::size_t> second_part = {};
56
57     for (std::size_t i = 0; i < n_count; i++) {
58         char action_type = '-';
59         std::cin >> action_type;
60         if (action_type == '-') {
61             PopGoblin(first_part, second_part);
62             continue;
63         }
64         std::size_t idx = 0;
65         std::cin >> idx;
66         AddGoblin(first_part, second_part, idx, action_type == '*');
67     }
68     return 0;
69 }

```

1.3 Менеджер памяти-1

1.3.1 Описание решения

В задаче необходимо реализовать прототип аллокатора. Воспользуемся красно-чёрными деревьями для хранения свободных блоков памяти в отсортированном виде. Каждый блок памяти представим в виде пары значений: начального адреса в памяти и его размера.

Пусть блоки памяти будут упорядочены во множестве по размеру, а в ассоциативном массиве — по начальному адресу. В момент запуска программы вся выделенная память доступна для аллокаций.

Если поступил запрос на аллокацию блока размера K , ищем во множестве минимальный по размеру свободный блок, который вмещает в себя K ячеек. Если такой не найден, то выводим ошибку аллокации. Иначе обновляем размер и начальный адрес выбранного свободного блока и вставляем обратно (*если осталось свободное место*). Аллоцированный блок данных сохраняем в хеш-таблицу по связке с номером запроса.

Если поступил запрос на деаллокацию блока, аллоцированного в i -м запросе, достаём его из хеш-таблицы и удаляем запись. Возвращаем его в ассоциативный массив и смотрим на соседние блоки. Если они примыкают вплотную, то производим *дефрагментацию*, то есть сливаем их в единый блок. Попутно синхронизируем записи во множестве свободных блоков.

Симуляция аллокатора завершена.

1.3.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $O(M \cdot \log M)$. В случае аллокации поиск наименьшего свободного блока памяти во множе-

стве занимает логарифмическое время. Удаление и вставка вершин в красно-чёрные деревья тоже имеют логарифмическую сложность.

В случае деаллокации поиск выделенного блока по номеру запроса в среднем выполняется за константное время. Другие используемые операции над красно-чёрными деревьями выполняются последовательно и за логарифмическое время.

Пространственная сложность решения — линейная $\mathcal{O}(M)$. В памяти хранятся данные о выделенных блоках в нескольких структурах для оптимизации времени доступа. Этих записей не может быть больше запросов на аллокацию блоков в каждой из структур. Промежуточные данные хранятся в константном количестве памяти.

1.3.3 Листинг кода

```
1 #include <cstddef>
2 #include <cstdint>
3 #include <cstdlib>
4 #include <iostream>
5 #include <iterator>
6 #include <map>
7 #include <set>
8 #include <unordered_map>
9
10 namespace {
11 struct Block {
12     std::size_t start;
13     std::size_t size;
14
15     bool operator<(const Block& other) const {
16         if (size != other.size) {
17             return size < other.size;
18         }
19         return start < other.start;
20     }
21 };
22
23 void Allocate(
24     std::set<Block>& free_heap,
25     std::map<std::size_t, std::size_t>& free_blocks,
26     std::unordered_map<std::size_t, Block>& query_map,
27     const std::size_t alloc_size,
28     const std::size_t query_id
29 ) {
30     auto iter = free_heap.lower_bound({0, alloc_size});
31     if (iter != free_heap.end()) {
32         const Block block = *iter;
33         free_heap.erase(iter);
34         free_blocks.erase(block.start);
35
36         const std::size_t remaining_size = block.size - alloc_size;
37         if (remaining_size > 0) {
```



```

38     const std::size_t new_start = block.start + alloc_size;
39     free_heap.insert({new_start, remaining_size});
40     free_blocks[new_start] = remaining_size;
41 }
42 query_map[query_id] = {block.start, alloc_size};
43 std::cout << block.start + 1 << "\n";
44 } else {
45     std::cout << 1 << "\n";
46 }
47 }
48
49 void Deallocate(
50     std::set<Block>& free_heap,
51     std::map<std::size_t, std::size_t>& free_blocks,
52     std::unordered_map<std::size_t, Block>& query_map,
53     const std::size_t query_id
54 ) {
55     auto dict_it = query_map.find(query_id);
56     if (dict_it != query_map.end()) {
57         const Block allocated_block = dict_it->second;
58         query_map.erase(dict_it);
59
60         auto insert_res = free_blocks.insert({allocated_block.start,
61             allocated_block.size});
62         auto curr_it = insert_res.first;
63
64         auto next_it = std::next(curr_it);
65         if (next_it != free_blocks.end() &&
66             allocated_block.start + allocated_block.size == next_it
67             ->first) {
68             free_heap.erase({next_it->first, next_it->second});
69
70             curr_it->second += next_it->second;
71             free_blocks.erase(next_it);
72         }
73
74         if (curr_it != free_blocks.begin()) {
75             auto prev_it = std::prev(curr_it);
76             if (prev_it->first + prev_it->second == curr_it->first) {
77                 free_heap.erase({prev_it->first, prev_it->second});
78
79                 prev_it->second += curr_it->second;
80                 free_blocks.erase(curr_it);
81
82                 curr_it = prev_it;
83             }
84         }
85         free_heap.insert({curr_it->first, curr_it->second});
86     }
87 }
88 } // namespace

```

```

87
88 int main() {
89     std::size_t n_count = 0;
90     std::size_t m_count = 0;
91     std::cin >> n_count >> m_count;
92
93     std::set<Block> free_heap;
94     std::map<std::size_t, std::size_t> free_blocks;
95     std::unordered_map<std::size_t, Block> query_map;
96     free_heap.insert({0, n_count});
97     free_blocks[0] = n_count;
98
99     for (std::size_t i = 0; i < m_count; i++) {
100         int32_t item = 0;
101         std::cin >> item;
102         if (item > 0) {
103             const auto alloc_size = static_cast<std::size_t>(item);
104             Allocate(free_heap, free_blocks, query_map, alloc_size, i
105                     + 1);
106             continue;
107         }
108         const std::size_t query_id = std::abs(item);
109         Deallocate(free_heap, free_blocks, query_map, query_id);
110     }
111     return 0;
112 }

```

1.4 Минимум на отрезке

1.4.1 Описание решения

В задаче необходимо реализовать поиск минимума в скользящем окне. Для этого введём список рекордов (*минимумов*) следующим образом:

1. Убираем все элементы с конца списка, которые больше текущего.
2. Добавляем текущий элемент в конец списка.
3. Убираем начальный элемент, если он вышел за границы текущего окна.

В конце каждой итерации выводим начальный элемент списка рекордов (*для текущего положения окна*).

1.4.2 Асимптотическая сложность

Временная сложность решения — линейная $O(N)$. Над каждым элементом массива произведено константное количество операций, которые, в свою очередь, тоже выполняются за константное время у двусвязного списка.

Пространственная сложность решения — линейная $O(N)$. Исходный массив материализуется, промежуточные значения занимают константное количество памяти.

1.4.3 Листинг кода

```
1 #include <cstddef>
2 #include <stdint>
3 #include <iostream>
4 #include <list>
5
6 struct EnumeratedItem {
7     std::size_t idx;
8     int32_t value;
9 };
10
11 namespace {
12 void DoStep(
13     std::list<EnumeratedItem>& window, const std::size_t
14     curr_idx, const std::size_t window_size
15 ) {
16     int32_t item = 0;
17     std::cin >> item;
18     while (!window.empty() && window.back().value >= item) {
19         window.pop_back();
20     }
21     window.push_back({curr_idx, item});
22     if (curr_idx - window.front().idx + 1 > window_size) {
23         window.pop_front();
24     }
25 }
26 // namespace
27
28 int main() {
29     std::size_t n_count = 0;
30     std::size_t k_count = 0;
31     std::cin >> n_count >> k_count;
32     if (k_count > n_count) {
33         std::cerr << "n_count must be greater than or equal to
34             k_count";
35         return 1;
36     }
37
38     std::list<EnumeratedItem> window = {};
39     for (std::size_t i = 0; i < k_count; i++) {
40         DoStep(window, i, k_count);
41     }
42     std::cout << window.front().value << " ";
43     for (std::size_t i = k_count; i < n_count; i++) {
44         DoStep(window, i, k_count);
45         std::cout << window.front().value << " ";
46     }
47     return 0;
48 }
```

2 Timus

2.1 War Games 2

2.1.1 Описание решения

В задаче необходимо определить последовательность солдат в порядке их убывания. Если реализовать решение «в лоб» (*через массив с удалением элементов*), то его сложность окажется квадратичной. Это решение не уложится по времени.

Другой вариант — заменить массив на связный список. В таком случае удаление происходит за константное время, но шаг через K солдат становится линейным. Имеем сложность $\mathcal{O}(N \cdot K)$, что тоже недопустимо.

Попробуем декомпозировать исходный массив на блоки длины $\lceil \sqrt{N} \rceil$. Тогда сложность поиска нужного солдата и его удаления суммарно равна $\mathcal{O}(\lceil \sqrt{N} \rceil)$. Из этого следует, что конечная сложность упала до $\mathcal{O}(N \cdot \sqrt{N})$. Этого достаточно, чтобы пройти ограничение по времени.

2.1.2 Асимптотическая сложность

Временная сложность решения — $\mathcal{O}(N \cdot \sqrt{N})$. Применяется принцип *корневой декомпозиции*.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. Материализуется массив последовательных чисел $1 \dots N$, который симулирует задачу. Промежуточные результаты занимают константное количество памяти.

2.1.3 Листинг кода

```
1 #include <cmath>
2 #include <cstdint>
3 #include <iostream>
4 #include <vector>
5
6 namespace {
7 struct SqrtBlock {
8     std::size_t sum;
9     std::vector<std::size_t> soldiers;
10 };
11
12 std::vector<SqrtBlock> InitCircle(std::size_t n_count) {
13     auto n_sqrt_c = static_cast<std::size_t>(std::ceil(std::sqrt(
14         n_count)));
15     std::vector<SqrtBlock> circle(n_sqrt_c, {0, {}});
16
17     std::size_t soldier_idx = 1;
18     for (std::size_t i = 0; i < n_sqrt_c; i++) {
19         while (soldier_idx <= n_count && circle[i].sum < n_sqrt_c) {
20             circle[i].soldiers.push_back(soldier_idx);
21             circle[i].sum++;
22             soldier_idx++;
23         }
24     }
25 }
```

```

23     }
24     return circle;
25 }
26
27 std::size_t FindAndRemove(std::vector<SqrtBlock>& circle, std::
    size_t pos) {
28     for (auto& block : circle) {
29         if (pos <= block.sum) {
30             const auto offset = static_cast<std::ptrdiff_t>(pos - 1);
31             const auto iter = block.soldiers.begin() + offset;
32             const std::size_t val = *iter;
33             block.soldiers.erase(iter);
34             block.sum--;
35             return val;
36         }
37         pos -= block.sum;
38     }
39     return 0;
40 }
41
42 void ProcessCircle(std::vector<SqrtBlock>& circle, std::size_t
    n_count, std::size_t k_count) {
43     std::size_t current_pos = k_count;
44     for (std::size_t m_count = n_count; m_count >= 1; m_count--) {
45         const std::size_t soldier_id = FindAndRemove(circle,
            current_pos);
46         std::cout << soldier_id << " ";
47
48         if (m_count > 1) {
49             current_pos = (current_pos + k_count - 1) % (m_count - 1);
50             if (current_pos == 0) {
51                 current_pos = (m_count - 1);
52             }
53         }
54     }
55 }
56 } // namespace
57
58 int main() {
59     std::size_t n_count = 0;
60     std::size_t k_count = 0;
61     std::cin >> n_count >> k_count;
62
63     std::vector<SqrtBlock> circle = InitCircle(n_count);
64     ProcessCircle(circle, n_count, k_count);
65     return 0;
66 }

```

2.2 White Streaks

2.2.1 Описание решения

В этой задаче требуется найти суммарное количество горизонтальных и вертикальных белых полос в произвольном прямоугольнике. Если её решать «в лоб» обходом каждой клетки прямоугольника, то мы не уложимся по времени выполнения алгоритма. Следовательно, нужно найти оптимизацию.

Заметим, что на ответ влияют лишь чёрные клетки. То есть количество белых клеток между двумя чёрными однозначно определено. Сфокусируемся на их обработке.

Отсортируем все чёрные клетки сначала по абсциссе, затем по ординате. При последовательном обходе такого массива чёрные клетки одной колонки будут соседними. Следовательно, количество промежутков между такими клетками равно количеству вертикальных полос в данной колонке. Пройдёмся по каждой колонке прямоугольника и посчитаем их.

Стоит заметить, что в таком случае краевые ситуации не учитываются (*к примеру, промежуток между верхней стороной прямоугольника и первой чёрной клеткой данной колонки, если он есть, не считается*). Для разрешения этой проблемы добавим в массив чёрных клеток фиктивные элементы, которые расположены по краям данного прямоугольника. Они не увеличат число белых полос, но позволят сохранить логику алгоритма.

Аналогично поступим с подсчётом горизонтальных белых полос: отсортируем чёрные клетки сначала по ординате, затем по абсциссе, и посчитаем число горизонтальных промежутков.

В описанном алгоритме есть недостаток: одиночные белые клетки, которые окружены со всех сторон чёрными клетками, учитываются дважды (*как горизонтальная и вертикальная полоса*). Это недопустимо, поэтому на этапе подсчёта вертикальных полос будем вести множество клеток, которые подозрительны на «одиночество»: вертикальное расстояние между соседними чёрными клетками должно равняться двум. Если на этапе подсчёта горизонтальных полос гипотеза подтвердилась, то учитываем клетку лишь один раз. Иначе это обычная горизонтальная полоса, которая ограничена сверху и снизу произвольными чёрными клетками.

2.2.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $\mathcal{O}((N + M + K) \cdot \log(N + M + K))$. В алгоритме дважды используется сортировка линейно-логарифмической сложности массива длины $2 \cdot (N + M + 2) + K$.

После каждой сортировки происходит линейный обход полученного массива. На каждой итерации выполняются операции не более чем логарифмической сложности (*в связи с использованием красно-чёрного дерева для учёта подозрительных клеток*).

Пространственная сложность решения — линейная $\mathcal{O}(N + M + K)$. Массив чёрных клеток реализован, к нему добавлены в явном виде граничные клетки данного прямоугольника. Промежуточные вычисления занимают константное количество памяти.

2.2.3 Листинг кода

```
1 #include <algorithm>
2 #include <cstdint>
3 #include <iostream>
4 #include <set>
5 #include <vector>
6
7 namespace {
8 struct Point {
9     std::size_t x;
10    std::size_t y;
11
12    bool operator<(const Point& other) const {
13        if (x != other.x) {
14            return x < other.x;
15        }
16        return y < other.y;
17    }
18 };
19
20 void AddBounds(std::vector<Point>& arr, const std::size_t
    x_bound, const std::size_t y_bound) {
21     for (std::size_t y_idx = 0; y_idx <= y_bound + 1; y_idx++) {
22         for (const std::size_t x_idx : {static_cast<std::size_t>(0),
            x_bound + 1}) {
23             arr.push_back({x_idx, y_idx});
24         }
25     }
26     for (std::size_t x_idx = 1; x_idx < x_bound + 1; x_idx++) {
27         for (const std::size_t y_idx : {static_cast<std::size_t>(0),
            y_bound + 1}) {
28             arr.push_back({x_idx, y_idx});
29         }
30     }
31 }
32
33 bool SortByY(const Point& lpt, const Point& rpt) {
34     if (lpt.y != rpt.y) {
35         return lpt.y < rpt.y;
36     }
37     return lpt.x < rpt.x;
38 }
39 } // namespace
40
41 int main() {
42     std::size_t m_count = 0;
43     std::size_t n_count = 0;
44     std::size_t k_count = 0;
45     std::cin >> m_count >> n_count >> k_count;
46 }
```

```

47 std::vector<Point> black_points(k_count);
48 black_points.reserve(k_count + 2 * (n_count + m_count + 2));
49 for (std::size_t i = 0; i < k_count; i++) {
50     std::cin >> black_points[i].x >> black_points[i].y;
51 }
52 AddBounds(black_points, m_count, n_count);
53
54 std::size_t white_streaks = 0;
55 std::set<Point> suspicious_points = {};
56 std::sort(black_points.begin(), black_points.end());
57 for (std::size_t i = 0; i < black_points.size() - 1; i++) {
58     const Point& lpt = black_points[i];
59     const Point& rpt = black_points[i + 1];
60     if (lpt.x != rpt.x) {
61         continue;
62     }
63     const auto diff = static_cast<std::ptrdiff_t>(rpt.y - lpt.y)
64         ;
65     if (diff == 1) {
66         continue;
67     }
68     if (diff == 2) {
69         suspicious_points.insert({lpt.x, lpt.y + 1});
70         continue;
71     }
72     white_streaks++;
73 }
74
75 std::sort(black_points.begin(), black_points.end(), SortByY);
76 for (std::size_t i = 0; i < black_points.size() - 1; i++) {
77     const Point& lpt = black_points[i];
78     const Point& rpt = black_points[i + 1];
79     if (lpt.y != rpt.y) {
80         continue;
81     }
82     const auto diff = static_cast<std::ptrdiff_t>(rpt.x - lpt.x)
83         ;
84     if (diff == 1) {
85         continue;
86     }
87     if (diff == 2) {
88         const Point sus_point = {lpt.x + 1, lpt.y};
89         if (suspicious_points.count(sus_point) != 0U) {
90             suspicious_points.erase(sus_point);
91             white_streaks++;
92         }
93         continue;
94     }
95     white_streaks++;
96 }
97 std::cout << white_streaks;

```



```
96     return 0;  
97 }
```